

United International University

CSE 2218: Data Structures and Algorithms Laboratory II

Assignment-2 on Dynamic Programming

Assignment: Problem 1-10

For final practice these problems along with
0/1 knapsack, [coin change min no of ways](#), [coin change max no of ways](#)

1. [Minimum Cost Climbing Stairs](#)
2. [Cut into Segments](#)
3. [Minimum Cost for Tickets](#)
4. [Climbing Stairs](#)
5. [Best Time to Buy and Sell Stocks](#)
6. [Minimum Path Sum](#)
7. [Unique Paths](#)
8. [House Robber](#)
9. [Maximum Sum of Non-Adjacent Elements](#)

You are given an array/list of 'N' integers. You are supposed to return the maximum sum of the subsequence with the constraint that no two elements are adjacent in the given array/list.

Note: A subsequence of an array/list is obtained by deleting some number of elements (can be zero) from the array/list, leaving the remaining elements in their original order.

Input format:

The first line contains a single integer 'T' denoting the number of test cases.

The first line of each test case contains a single integer 'N' denoting the number of elements in the array.

The second line contains 'N' single space-separated integers denoting the elements of the array/list.

Output format:

For each test case, print a single integer that denotes the maximum sum of the non-adjacent elements.

Print the output of each test case in a separate line.

Constraints:

$1 \leq T \leq 500$

$1 \leq N \leq 1000$

$0 \leq \text{ARR}[i] \leq 105$

Where 'ARR[i]' denotes the ith element in the array/list

Sample Input	Sample output
2 3 1 2 4 4 2 1 4 9	5 11
2 5 1 2 3 5 4 9 1 2 3 1 3 5 8 1 9	8 24

Explanation to Sample Output 1:

In test case 1, the sum of 'ARR[0]' & 'ARR[2]' is 5 which is greater than 'ARR[1]' which is 2 so the answer is 5.

In test case 2, the sum of 'ARR[0]' and 'ARR[2]' is 6, the sum of 'ARR[1]' and 'ARR[3]' is 10, and the sum of 'ARR[0]' and 'ARR[3]' is 11. So if we take the sum of 'ARR[0]' and 'ARR[3]', it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.

Explanation to Sample Output 2:

In test case 1, out of all the possibilities, if we take the sum of 'ARR[0]', 'ARR[2]' and 'ARR[4]', i.e. 8, it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.

In test case 2, out of all the possibilities, if we take the sum of 'ARR[0]', 'ARR[2]', 'ARR[4]', 'ARR[6]' and 'ARR[8]', i.e. 24 so, it will give the maximum sum of sequence in which no elements are adjacent in the given array/list.

10. You are given a target amount M that needs to be paid exactly.

There are N available coins, and the value of each coin is given. Each coin can be used at most once. However, there is also a restriction: you are allowed to use **no more than D coins**.

Your task is to determine whether it is possible to choose some of the given coins such that their total value becomes exactly M and the number of chosen coins does not exceed D .

Print "YES" if it is possible. Otherwise, print "NO".

Input Format

M N

V_1 V_2 V_3 ... V_N

D

Where:

M = target amount

N = number of coins

V_i = value of the i -th coin

D = maximum number of coins allowed

Output Format

YES

if the exact amount can be paid using at most D coins.

Otherwise, print:

NO

Sample Input	Sample output
100 5 25 25 25 25 50 2	NO
100 5 25 25 25 25 50 4	YES

